

# Code-Layout, Readability and Reusability

|               |                   |
|---------------|-------------------|
| Date          | 19 September 2026 |
| Team ID       | SWTID-2026-9627   |
| Project Name  | AI Fit Track API  |
| Maximum Marks | 2 Marks           |

## Code Layout, Readability and Reusability

AI Fit Track follows a modular MVC-inspired layout under `src/`, keeping every concern in its own folder so a new route can be added without touching unrelated files.

| Folder       | Purpose                                                                                      | Reusability Practice                                                                                                                                             |
|--------------|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| config/      | MongoDB connection setup ( <code>db.js</code> )                                              | Connection logic centralized once; every model imports the same connected instance                                                                               |
| controllers/ | Request-handling logic per resource (auth, workout, AI)                                      | Thin controllers delegate persistence to Models and AI calls to <code>services/</code> , so logic isn't duplicated                                               |
| middleware/  | JWT auth guard, centralized error handler                                                    | A single <code>auth.js</code> is reused across every protected route instead of re-checking tokens per controller                                                |
| models/      | Mongoose schemas ( <code>User.js</code> , <code>Workout.js</code> ) with validation baked in | Validation rules (required, enum, min) live once in the schema, not repeated in every controller                                                                 |
| routes/      | Endpoint-to-controller mapping per resource                                                  | Each route file is mounted with its own prefix ( <code>/api/auth</code> , <code>/api/workouts</code> , <code>/api/ai</code> ), keeping URL structure predictable |
| services/    | <code>geminiService.js</code> – isolates all Google Gemini prompt/response handling          | Both AI endpoints (recommendation, insights) call the same service module with different prompt builders, avoiding duplicated Gemini client setup                |

Consistent naming (camelCase functions, PascalCase models), small single-purpose functions per controller action, and comments above non-obvious logic (e.g., the Gemini prompt templates) keep the codebase readable for all 5 contributors working in parallel.